RESEARCH PROJECT REPORT

DRIFTWATCH: A COMPARATIVE STUDY OF DRIFT DETECTION AND AUTOMATED RETRAINING IN AN MLOPS PIPELINE

Course: MLOps (AIM4142), Assignment for Fast Learners

Submitted in partial fulfilment
of the
requirement for the award of the degree of

BACHELOR OF TECHNOLOGY (B. Tech)

in

Computer Science and Engineering

Submitted by

Anushka Srivastava

(Registration No.)

Submitted to

Name of Faculty Instructor



MANIPAL UNIVERSITY
JAIPUR
(University under Section 2(f) of the UGC Act)

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING,
SCHOOL OF COMPUTER SCIENCE & ENGINEERING,
MANIPAL UNIVERSITY JAIPUR,
RAJASTHAN, INDIA-303007

OCTOBER, 2026

Date: _____

CERTIFICATE

This is to certify that Ms. Anushka Srivastava (Registration No.) has completed the research project titled “DriftWatch: A Comparative Study of Drift Detection and Automated Retraining in an MLOps Pipeline” as part of the course MLOps (AIM4142), under the Assignment for Fast Learners, during the period from _____ to 30 October 2026, as a partial fulfilment of the degree of Bachelor of Technology in Computer Science and Engineering of Manipal University Jaipur, during the academic year 2026-27. The work was carried out in the field of Machine Learning Operations, specifically model drift detection, model monitoring and automated retraining.

Signature of Faculty Instructor
Name of Faculty Instructor
Department of Artificial Intelligence and Machine Learning
Manipal University Jaipur

DECLARATION

I hereby declare that the research project report on "DriftWatch: A Comparative Study of Drift Detection and Automated Retraining in an MLOps Pipeline" is an authentic record of my own work, carried out as a requirement of the course MLOps (AIM4142) during the period from _____ to 30 October 2026, for the award of the degree of B.Tech. (Computer Science and Engineering), Manipal University Jaipur, Rajasthan, under the guidance of Name of Faculty Instructor.

(Signature of student)
Anushka Srivastava
Registration No.

Date: _____

Certified that the above statement made by the student is correct to the best of our knowledge and belief.

Examined by:

(Signature)
Name of MUJ Faculty Instructor

(Signature)
Head of Department

ACKNOWLEDGMENTS

First and foremost, I wish to express my sincere thanks and gratitude to my course instructor, Name of Faculty Instructor, for offering the Assignment for Fast Learners in MLOps (AIM4142) and for the guidance and encouragement that shaped the direction of this research project.

I would also like to tender my sincere thanks to Prof. Name of HOD (Head of the Department of Artificial Intelligence and Machine Learning) and to the faculty members of the School of Computer Science and Engineering, Manipal University Jaipur, whose teaching laid the foundation for this work, for their co-operation and encouragement.

Finally, I am grateful to the open-source communities behind scikit-learn, MLflow, FastAPI and the wider Python ecosystem, whose tools made it possible to build and evaluate a complete MLOps pipeline within the time available.

(Signature of student)
Anushka Srivastava
Registration No.

ABSTRACT

Machine learning models deployed in production are trained on historical data, but the data they see after deployment keeps changing. When the input distribution or the relationship between inputs and outcomes shifts, a phenomenon known as drift, model accuracy decays silently. MLOps teams must therefore decide how to detect drift and when to retrain: retraining too rarely loses accuracy, while retraining too often wastes compute and risks deploying worse models. The objective of this project is to compare drift detection strategies inside a complete, automated MLOps pipeline and to identify which strategy detects each type of drift fastest, with the fewest false alarms, and with the best accuracy recovery.

A simulated loan-default production stream of 60 batches of 500 applications was built with controlled drift injected at batch 30 under four scenarios: no drift, sudden covariate drift, gradual covariate drift and concept drift. Five detectors were evaluated: the Population Stability Index (PSI), the Kolmogorov–Smirnov (KS) test, Jensen–Shannon (JS) distance, Maximum Mean Discrepancy (MMD) and the Drift Detection Method (DDM), together with two baselines, no retraining and periodic retraining. Each alarm triggers an automated retraining job whose challenger model is promoted only if it beats the current champion on the newest data. In total, 140 production streams were simulated (4 scenarios × 7 strategies × 5 random seeds).

Feature-distribution detectors (PSI and JS) never detected concept drift, leaving accuracy at 62.9%, identical to never retraining. The performance-based DDM detector caught concept drift in every run within 0.2 batches on average, with zero false alarms, and restored post-drift accuracy to 83.3%, the best of all strategies. Covariate drift was flagged immediately by all feature detectors, but retraining improved accuracy by less than one percentage point, showing that not all drift requires retraining. The KS and MMD tests caused 7.2 and 4.4 unnecessary retraining jobs per stream when no drift was present. These results support a hybrid monitoring design: label-free distribution tests for early warning and performance monitoring to decide when to retrain.

The system was implemented in Python using scikit-learn for modelling, NumPy and SciPy for the statistical tests, MLflow for experiment tracking and the model registry, FastAPI for model serving and drift-check endpoints, Matplotlib for visualisation, pytest for testing, GitHub Actions for continuous integration and Docker for containerisation.

CONTENTS

			Page No.
Declaration			i
Acknowledgments			ii
Abstract			iii
List of Figures			v
List of Tables			vi
Chapter 1	**INTRODUCTION**		1
	1.1	Motivation	1
Chapter 2	**LITERATURE REVIEW**		3
	2.1	MLOps and the Problem of Model Decay	3
	2.2	Types of Drift and Drift Detection Methods	3
	2.3	Outcome of Literature Review	4
	2.4	Problem Statement	5
	2.5	Research Objectives	5
Chapter 3	**METHODOLOGY AND FRAMEWORK**		6
	3.1	System Architecture	6
	3.2	Algorithms and Techniques	6
	3.3	Detailed Design Methodology	8
Chapter 4	**WORK DONE**		11
	4.1	Implementation Details	11
	4.2	Results and Discussion	15
	4.3	Individual Contribution	18
Chapter 5	**CONCLUSION AND FUTURE WORK**		19
	5.1	Proposed Work Plan of the Project	19
References			21
Annexures			23

LIST OF FIGURES

Figure No.	Figure Title	Page No.
3.1	System architecture of DriftWatch	6
3.2	Effect of the injected drift on feature distributions (left, centre) and on the default rate (right)	9
4.1	Live demo output: concept drift detected by DDM and model recovery through two promotions	12
4.2	MLflow experiment tracking: benchmark runs with per-run accuracy and summary metric charts	13
4.3	FastAPI service documentation (Swagger UI)	14
4.4	/drift/simulate responses: sudden covariate drift is flagged (top), concept drift is not (bottom)	14
4.5	Mean model accuracy per batch under the four drift scenarios	15
4.6	Detection delay and hit rate per detector and scenario	16
4.7	Mean accuracy after the drift point for each strategy and scenario	17
4.8	Unnecessary retraining jobs per stream in the no-drift control	18
A.1	MLflow runs table for the driftwatch-benchmark experiment	23
A.2	MLflow Model Registry: promoted models registered as versions of loan-default-model	24

LIST OF TABLES

Table No.	Table Title	Page No.
2.1	Summary of key literature reviewed	4
3.1	Drift detectors and alarm rules	8
3.2	Drift scenarios	8
3.3	Experimental parameters	10
4.1	Development environment and tools	11
4.2	Source modules	12
4.3	API endpoints	13
4.4	Detection performance (detection rate / mean delay in batches) and false alarms	16
4.5	Mean accuracy after drift per scenario, concept-drift F1-score and wasted retrains	17
5.1	Proposed work plan	20

CHAPTER 1
INTRODUCTION

Machine learning has moved from research notebooks into production systems that approve loans, detect fraud, recommend products and forecast demand. Deploying a model, however, is not the end of its life cycle. A model is a snapshot of the data it was trained on, and the world that generates new data does not stand still. Customer behaviour changes, economic conditions shift, upstream data pipelines are modified and new user segments appear. As a result, a model that performed well at deployment can quietly lose accuracy over the following weeks or months, often without any visible error being raised.

Machine Learning Operations (MLOps) is the discipline that applies DevOps principles of automation, versioning, continuous integration and monitoring to the machine learning life cycle [1], [2]. One of its central concerns is keeping deployed models reliable over time. This requires three capabilities working together: monitoring to notice that something has changed, decision logic to determine whether the change matters, and automated retraining with safeguards to fix the problem without introducing new ones.

This project, **DriftWatch**, builds such a pipeline end to end and uses it as an experimental platform. It compares five widely used drift detectors and two baseline retraining policies under controlled drift scenarios, measuring how quickly each strategy detects drift, how often it raises false alarms, how much retraining it triggers and how well it restores model accuracy.

1.1 Motivation

Surveys of real-world machine learning deployments consistently identify monitoring and model maintenance as among the hardest operational challenges [3]. Sculley et al. described machine learning systems as carrying “hidden technical debt”, much of which arises from the dependency of model behaviour on external data that can change at any time [1]. A model whose inputs drift may continue to return predictions with full confidence while being systematically wrong.

In practice, organisations take one of three approaches. Some deploy and forget, which is cheap but leaves models exposed to silent decay. Others retrain on a fixed schedule, for example every week, which is simple but blind: it retrains when nothing has changed and may wait too long when something has. A third approach is to monitor data and model behaviour and retrain only when drift is detected. Many tools, such as Evidently AI, provide drift detectors, but the choice of detector is often made by convention rather than evidence. Practitioners are rarely told which detector suits which kind of drift, what false-alarm rate to expect, or whether detecting drift actually leads to better accuracy after retraining.

The motivation for this project is to answer these questions empirically within a realistic MLOps workflow. The project is relevant to any domain where models face changing data. Credit risk scoring, used here as the running example, is a particularly natural fit because economic shocks and policy changes are common causes of drift, and because the Population Stability Index, one of the detectors studied, originated in credit-risk monitoring [11].

Applications: the findings apply to production monitoring in finance, e-commerce, healthcare and any system where models are periodically retrained. **Advantages** of the approach include a quantified comparison of detectors under identical conditions, a reusable open-source pipeline, and a quality gate that prevents an automated retraining job from ever promoting a worse model.

CHAPTER 2
LITERATURE REVIEW

This chapter reviews the literature on MLOps and model decay, the types of drift and the main families of drift detectors, and then identifies the gap that this project addresses.

2.1 MLOps and the Problem of Model Decay

Kreuzberger et al. define MLOps as a paradigm that combines machine learning, software engineering and data engineering to productionise models reliably, and identify continuous monitoring and continuous training as two of its core principles [2]. Google's reference architecture for MLOps describes maturity levels ranging from manual deployment (level 0) to fully automated pipelines in which retraining is triggered by new data or by monitoring signals (levels 1 and 2) [21]. Breck et al. proposed the ML Test Score, a rubric for production readiness that explicitly includes monitoring for changes in input data and in prediction quality [17].

Paleyes et al. surveyed case studies of deployed machine learning and found that monitoring and updating models after deployment are recurring sources of failure [3]. Experiment-tracking platforms such as MLflow [15] address part of this problem by recording parameters, metrics and model versions, so that each retrained model can be traced and rolled back.

2.2 Types of Drift and Drift Detection Methods

Moreno-Torres et al. provide a unifying view of dataset shift in classification [6]. If X denotes the inputs and y the target, the joint distribution P(X, y) can change in different ways. **Covariate shift** occurs when P(X) changes but P(y|X) stays the same; for example, applicants become poorer, but the rule linking their characteristics to default is unchanged. **Concept drift** occurs when P(y|X) changes; for example, a new lending policy alters who defaults even though applicants look the same [4], [5]. Webb et al. further characterise drift by its duration and speed, distinguishing **sudden** from **gradual** drift [16].

Drift detectors fall into two broad families [5]. **Distribution-based detectors** compare the distribution of recent inputs with a reference sample, usually the training data. They need no labels, which makes them attractive in production, where true outcomes often arrive late. Examples include the Population Stability Index [11], the Kolmogorov–Smirnov test [12], the Jensen–Shannon divergence [13] and the kernel Maximum Mean Discrepancy test [9]. Rabanser et al. compared several such tests and found that simple univariate tests with multiple-testing correction perform competitively with more complex multivariate tests for detecting covariate shift [10].

Error-rate-based detectors monitor the performance of the model itself. The Drift Detection Method (DDM) of Gama et al. models the classification error as a binomial process and raises

an alarm when the error rate exceeds its historical minimum by three standard deviations [7]. Variants include EDDM, which tracks the distance between errors [20], and ADWIN, which maintains an adaptive window [8]. These detectors respond directly to loss of accuracy but require ground-truth labels.

Table 2.1: Summary of key literature reviewed

Work	Approach	Key finding	Limitation / gap
Sculley et al. [1]	Analysis of ML system maintenance costs	Data dependencies and changing external conditions create hidden technical debt	No specific detection method
Kreuzberger et al. [2]	Literature review and interviews on MLOps	Defines monitoring and continuous training as core MLOps components	Conceptual; no quantitative comparison
Gama et al. [4]	Survey of concept drift adaptation	Taxonomy of drift types and detectors; recommends evaluation by delay and false alarms	Focus on streaming learners, not MLOps pipelines
Lu et al. [5]	Review of learning under concept drift	Distinguishes error-rate, distribution and multiple-hypothesis detectors	Limited guidance on retraining cost
Gama et al. [7]	DDM: error-rate monitoring with control limits	Detects concept drift from rising classification error	Requires labels; sensitive to delay in label arrival
Gretton et al. [9]	Kernel two-sample test (MMD)	Detects any difference in multivariate distributions	Computational cost; needs permutation tests
Rabanser et al. [10]	Empirical comparison of dataset-shift detectors	Two-sample tests on model representations work well for covariate shift	Does not study concept drift or retraining outcomes
Yurdakul [11]	Statistical properties of PSI	Explains PSI thresholds used in credit scoring	Univariate; insensitive to P(y|X) changes

2.3 Outcome of Literature Review

The review leads to four observations:

1. Distribution-based and error-based detectors observe different quantities, P(X) and model error respectively, so neither family can be expected to detect every type of drift. However, this is rarely quantified side by side within the same pipeline.
2. Most comparative studies evaluate detectors in isolation using detection delay and false alarms [4], [10], but do not measure the downstream outcome that matters in MLOps: whether retraining after an alarm actually restores accuracy, and at what compute cost.
3. Fixed-schedule (periodic) retraining is widely used in industry but is seldom included as a baseline in drift-detection studies.
4. Few studies embed detectors in a pipeline with the safeguards used in practice, such as champion/challenger validation, a model registry and automated promotion.

2.4 Problem Statement

Given a classification model deployed in a production environment where data can drift, determine which drift detection strategy, among PSI, KS, JS, MMD and DDM, together with no-retraining and periodic-retraining baselines, should trigger automated retraining so as to (a) detect harmful drift quickly, (b) avoid unnecessary retraining when no harmful drift is present, and (c) maximise model accuracy after drift, for both covariate and concept drift.

2.5 Research Objectives

1. To design and implement an automated MLOps pipeline that serves a model, monitors incoming data, detects drift, retrains, validates challengers against the champion and promotes better models.
2. To build a controlled data stream with four drift scenarios (no drift, sudden covariate, gradual covariate and concept drift) so that the exact onset of drift is known.
3. To benchmark five detectors and two baselines using detection rate, detection delay, false alarms, number of retraining jobs and post-drift accuracy over multiple random seeds.
4. To integrate production MLOps tooling, namely MLflow experiment tracking and model registry, a FastAPI serving layer with drift-check endpoints, automated tests, CI and containerisation.
5. To derive evidence-based recommendations for choosing drift-monitoring strategies in practice.

CHAPTER 3
METHODOLOGY AND FRAMEWORK

This chapter describes the architecture of DriftWatch, the algorithms used for drift detection and modelling, and the detailed design of the data stream, retraining policy and evaluation protocol.

3.1 System Architecture

DriftWatch follows a closed-loop MLOps architecture with three layers (Figure 3.1). In the **production serving and monitoring layer**, each incoming batch is scored by the current champion model, and the monitoring layer runs drift tests on the batch. Feature tests compare the batch's inputs with the model's training data, while the performance test (DDM) runs on the model's errors once true labels arrive. If an alarm is raised, the **automated retraining layer** trains a challenger on the most recent labelled data, validates both models on the newest data, and promotes the challenger only if it is at least as accurate. The **MLOps platform layer** records every run in MLflow, registers each promoted model as a new version in the MLflow Model Registry, serves the current champion through a FastAPI service, and runs automated tests and a benchmark in GitHub Actions on every code change.

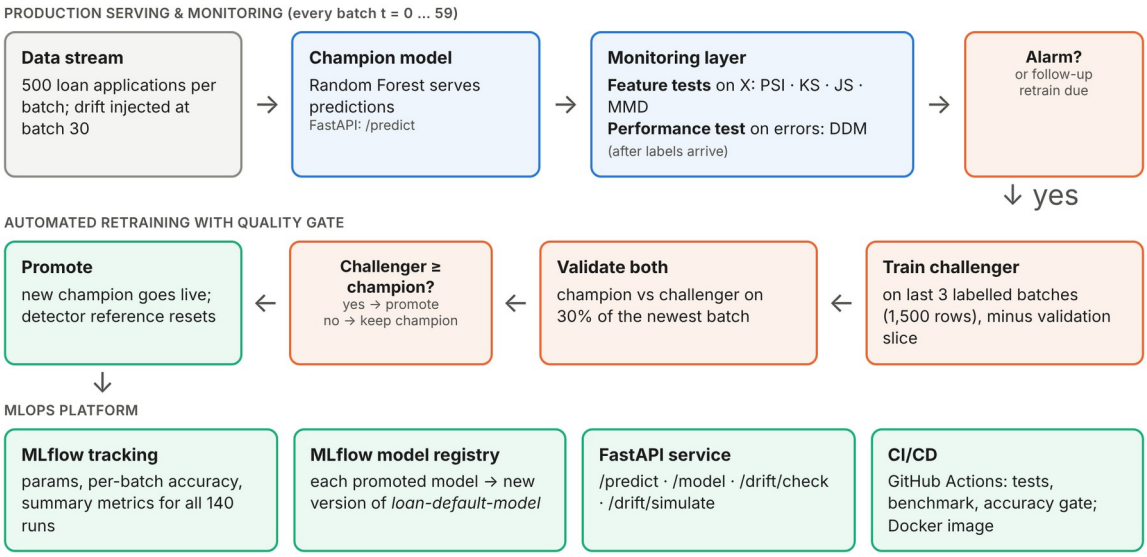


Figure 3.1: System architecture of DriftWatch

3.2 Algorithms and Techniques

Five detectors were implemented behind a common interface. Each detector receives a reference sample (the current model's training data) and a new batch, and returns an alarm flag, a score and the feature most responsible. Table 3.1 summarises the alarm rules.

3.2.1 Population Stability Index (PSI)

PSI is the standard stability metric in credit scoring [11]. Each feature is divided into ten quantile bins computed on the reference data. If p_i and q_i are the proportions of reference and new observations in bin i, then $PSI = \Sigma_i (q_i - p_i) \cdot \ln(q_i / p_i)$. By industry convention, PSI below 0.1 indicates no significant change, 0.1-0.2 a moderate change, and above 0.2 a significant shift. An alarm is raised if any feature exceeds 0.2.

3.2.2 Kolmogorov–Smirnov Test (KS)

The two-sample KS test compares the empirical cumulative distribution functions of a feature in the reference and new samples; its statistic is the maximum vertical distance between them [12]. Because one test is run per feature (six features), a Bonferroni correction is applied: an alarm is raised if any p-value falls below $0.05/6 \approx 0.0083$, which keeps the family-wise false-alarm probability per batch near 5%.

3.2.3 Jensen–Shannon Distance (JS)

The Jensen–Shannon divergence is a symmetric, bounded version of the Kullback–Leibler divergence [13]. For binned distributions P and Q with mixture $M = \frac{1}{2}(P + Q)$, $JS(P, Q) = \frac{1}{2} KL(P\|M) + \frac{1}{2} KL(Q\|M)$, and the JS distance is its square root. An alarm is raised if the distance for any feature exceeds 0.1, the default threshold Evidently AI applies to its distance-based drift tests [18].

3.2.4 Maximum Mean Discrepancy (MMD)

Unlike the three univariate tests above, MMD compares the full joint distribution of all six features [9]. Features are standardised using reference statistics, and an RBF kernel with bandwidth set by the median heuristic is used. The squared MMD between samples A and B is $MMD^2 = mean\ k(a, a') + mean\ k(b, b') - 2 \cdot mean\ k(a, b)$. Significance is assessed with a permutation test: the pooled sample is shuffled 50 times to build a null distribution, and an alarm is raised if the permutation p-value is below 0.05. Up to 250 points per sample are used to bound computation.

3.2.5 Drift Detection Method (DDM)

DDM monitors the model's running error rate p over n labelled predictions, with standard deviation $s = \sqrt{p(1 - p)/n}$ [7]. It records the minimum of p + s observed so far (p_min, s_min) and raises an alarm when $p + s > p_min + 3 \cdot s_min$. Because it reacts only to a rise in error, DDM is insensitive to input changes that do not hurt the model. Its statistics are reset whenever a new model is promoted.

3.2.6 Prediction Model

The deployed model is a Random Forest classifier [14] with 100 trees and maximum depth 8, implemented in scikit-learn [19]. Random forests are a common choice for tabular credit-risk

data, and they cannot extrapolate beyond the range of their training data. This makes the effect of drift, and the value of retraining, clearly visible.

Table 3.1: Drift detectors and alarm rules

Detector	Monitors	Alarm rule	Needs labels	Ref.
PSI	Distribution, per feature	Max PSI across features > 0.2	No	[11]
KS	Distribution, per feature	Min p-value < 0.05/6 (Bonferroni)	No	[12]
JS	Distribution, per feature	Max JS distance > 0.1	No	[13]
MMD	Distribution, multivariate	Permutation p-value < 0.05	No	[9]
DDM	Model error rate	p + s > p_min + 3·s_min	Yes	[7]

3.3 Detailed Design Methodology

3.3.1 Data Stream with Controlled Drift

Measuring detection delay requires knowing exactly when drift begins. Real-world datasets do not record this, so drift research commonly relies on generators with injected drift [4], [5]. DriftWatch simulates loan applications with six features: age, annual income, loan amount, credit score, debt ratio and years of employment. Income is log-normal, credit score is normal and clipped to 300-850, debt ratio follows a Beta distribution, and loan amount is a fraction of income. The probability of default is a logistic function of debt ratio, loan-to-income ratio and credit score, including a penalty for scores below 600. The base default rate is about 30%.

Each production stream consists of 60 batches of 500 applications. Drift begins at batch 30 according to one of the four scenarios in Table 3.2. Figure 3.2 shows how the economic shock moves the income and debt-ratio distributions and how each scenario changes the share of applicants who default.

Table 3.2: Drift scenarios

Scenario	Description	Changes	Speed
No drift (control)	Stationary stream; any alarm is a false alarm	None	-
Sudden covariate	Economic shock at batch 30: median income −22%, loan amounts +20%, mean credit score −40, mean debt ratio 0.29 → 0.41	P(X)	Sudden
Gradual covariate	The same shock phased in linearly over batches 30-49	P(X)	Gradual
Concept	New lending regime at batch 30: credit score barely matters; age < 32, short employment and loan-to-income dominate default risk. Inputs unchanged	P(y|X)	Sudden

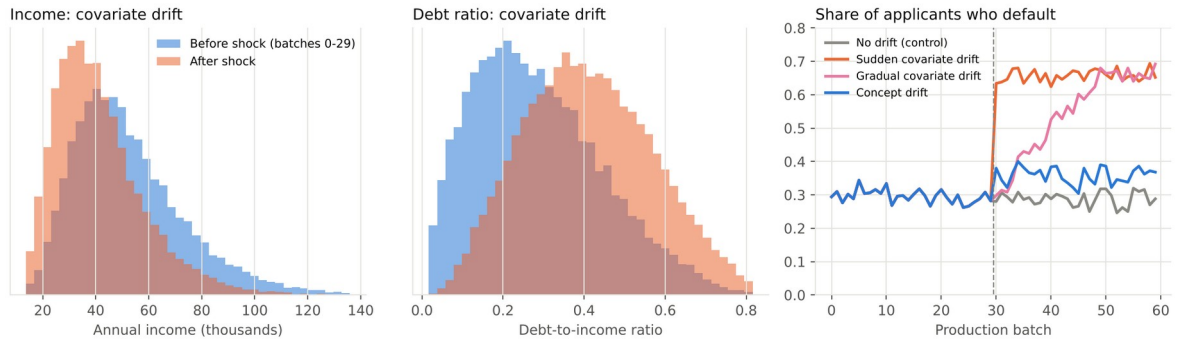


Figure 3.2: Effect of the injected drift on feature distributions (left, centre) and on the default rate (right)

3.3.2 Retraining Policy and Quality Gate

For every batch, the champion first predicts, after which the true labels are revealed. This assumes labels eventually become available, as when loans are later repaid or not. The monitoring strategy then decides whether to retrain. On retraining, a challenger is trained on the last three labelled batches, excluding a stratified 30% slice of the newest batch held out for validation. Both champion and challenger are scored on that slice, and the challenger is promoted only if its accuracy is at least that of the champion. Validating on the newest batch is essential: in an early version of the pipeline, validation used the whole window, which after concept drift was two-thirds old-regime data. That caused better challengers to be rejected.

When a detector raises an alarm, the retraining window still contains pre-drift batches. A **follow-up retrain** is therefore scheduled two batches later, when the window holds only post-alarm data. After any detector-triggered retrain, the detector's reference sample is reset to the new training window, so a drift that has been acknowledged does not trigger alarms indefinitely. The periodic baseline retrains at batches 5, 15, 25, 35, 45 and 55. This schedule was chosen so that its first post-drift retrain occurs five batches after onset, its average-case delay, rather than coinciding by chance with the drift point.

3.3.3 Evaluation Metrics and Experimental Protocol

Each of the seven strategies was run on each of the four scenarios with five random seeds, giving 140 runs. The following metrics were recorded:

- **Detection rate**: the fraction of runs in which an alarm fires within 10 batches of drift onset.
- **Detection delay**: the number of batches between drift onset and the first alarm, for detected runs.
- **False alarms per 100 batches**: alarms raised before drift onset, or at any time in the no-drift control.
- **Retraining jobs**: the total number of retrains per stream, a proxy for compute cost.

- **Post-drift accuracy and F1-score**: the mean over batches 30-59, measuring how well the strategy protects model quality.

Table 3.3: Experimental parameters

Parameter	Value
Production stream	60 batches × 500 applications; drift starts at batch 30
Initial training set	3,000 historical (pre-drift) applications
Model	Random Forest, 100 trees, max depth 8 [14]
Retraining window	Last 3 labelled batches (1,500 rows)
Validation	30% of the newest batch (stratified), held out from training
Promotion rule	Promote challenger if accuracy ≥ champion on validation slice
Follow-up retrain	2 batches after each detector alarm
Periodic baseline	Retrain at batches 5, 15, 25, 35, 45, 55
Detection window	Alarm within 10 batches of drift onset counts as detected
Repetitions	5 random seeds per (scenario, strategy) pair; 140 runs in total

CHAPTER 4
WORK DONE

This chapter describes the implementation of DriftWatch and presents and discusses the experimental results.

4.1 Implementation Details

4.1.1 Development Environment

The project was developed in Python and uses only open-source tools (Table 4.1). It runs on an ordinary Windows laptop: a set of batch launchers creates a virtual environment, installs dependencies inside the project folder, runs the demo and benchmark, and opens the results report.

Table 4.1: Development environment and tools

Component	Technology
Language	Python 3.10+ (developed and tested on Python 3.13)
ML and statistics	scikit-learn 1.9, NumPy, SciPy, pandas
Experiment tracking and registry	MLflow 3 (SQLite backend store)
Model serving	FastAPI with Uvicorn; Swagger UI documentation
Visualisation	Matplotlib; HTML results report
Testing and CI	pytest; GitHub Actions workflow
Containerisation	Docker (python:3.11-slim base image)
Parallel execution	joblib (runs distributed across all CPU cores)
IDE and OS	Visual Studio Code on Windows; one-click .bat launchers

4.1.2 Source Modules

The code is organised into small modules with single responsibilities (Table 4.2). All detectors share one interface, *set_reference(X)* and *check(X, y_true, y_pred)*, so new detectors can be added without changing the pipeline.

Table 4.2: Source modules

Module	Responsibility
src/data.py	Data generator and the four drift scenarios
src/detectors.py	PSI, KS, JS, MMD and DDM behind a common interface
src/pipeline.py	Serve → monitor → retrain → validate → promote loop
src/experiment.py	140-run benchmark, metrics, charts, HTML report, MLflow logging
src/demo.py	Live single-stream demo with MLflow model registration
api/main.py	FastAPI service exposing prediction and drift endpoints
tests/	Unit and integration tests
.github/workflows/ci.yml	CI: tests, quick benchmark and accuracy quality gate

4.1.3 Live Pipeline Demonstration

The demo module runs one stream and prints the pipeline's decisions batch by batch. Figure 4.1 shows the concept-drift scenario monitored by DDM. Accuracy is stable at about 0.88-0.89 until batch 30, when the new regime causes it to collapse to 0.61. DDM raises an alarm in the same batch. The first challenger, trained on a window still dominated by old data, is only slightly better but is promoted (0.593 vs 0.540). The follow-up retrain at batch 32 uses only post-drift data and is promoted with 0.887 validation accuracy against the champion's 0.713, after which accuracy stabilises around 0.85.

```
$ python -m src.demo

=== DriftWatch live demo | scenario: concept | monitoring: DDM ===
Drift is injected at batch 30. Watch accuracy, alarms, retraining and promotions.
batch 26 | acc 0.890 | ok
batch 27 | acc 0.890 | ok
batch 28 | acc 0.882 | ok
batch 29 | acc 0.890 | ok
batch 30 | acc 0.612 | ALARM (DDM: error rate) -> challenger 0.593 vs champion 0.540 -> PROMOTED to v2
batch 31 | acc 0.636 | ok
batch 32 | acc 0.672 | follow-up retrain -> challenger 0.887 vs champion 0.713 -> PROMOTED to v3
batch 33 | acc 0.856 | ok
batch 34 | acc 0.870 | ok
batch 35 | acc 0.856 | ok
batch 36 | acc 0.844 | ok
batch 37 | acc 0.840 | ok
batch 38 | acc 0.872 | ok
...
--- summary ---
detection_delay  0
false_alarms     0
retrains         2
promotions       2
acc_pre_drift    0.876
acc_post_drift   0.831
acc_last_10      0.851
```

Figure 4.1: Live demo output: concept drift detected by DDM and model recovery through two promotions

4.1.4 Experiment Tracking and Model Registry

Every benchmark run is logged to MLflow with its parameters (scenario, strategy, seed, window size), its per-batch accuracy and model version as step-wise metrics, and its summary metrics. This allows any run to be inspected and compared in the MLflow user interface (Figure 4.2). In the live demo, every promoted model is logged with an input example and registered as a new version of *loan-default-model* in the MLflow Model Registry (Figure A.2 in the Annexures), providing a versioned, auditable history of the models that went live.

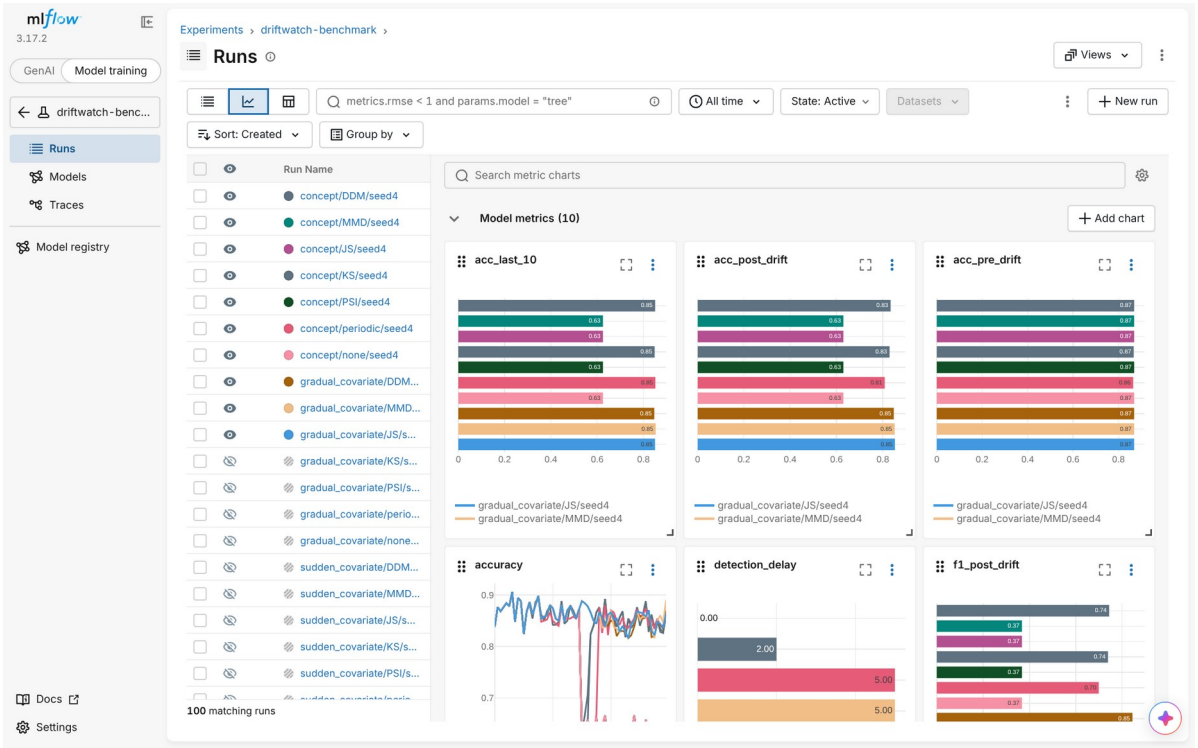


Figure 4.2: MLflow experiment tracking: benchmark runs with per-run accuracy and summary metric charts

4.1.5 Model Serving API

The current champion is served by a FastAPI application (Table 4.3, Figure 4.3). Besides prediction, the API exposes monitoring endpoints. */drift/check* accepts a batch of recent applications and returns the verdict of the four label-free detectors, the score of each and the most shifted feature. The batch is reported as drifted when at least two detectors agree. */drift/simulate* generates a batch from a chosen scenario for demonstration. Figure 4.4 contrasts the two drift types: for sudden covariate drift all four detectors flag the batch, with debt ratio as the most shifted feature, whereas for concept drift none of them do, because the inputs have not changed.

Table 4.3: API endpoints

Endpoint	Purpose
GET /health	Service and model availability
GET /model	Version and metadata of the current champion
POST /predict	Default probability for one or more applicants
POST /drift/check	Runs PSI, KS, JS and MMD on a submitted batch against the model's training data
GET /drift/simulate	Generates a batch from a chosen scenario and returns the drift report

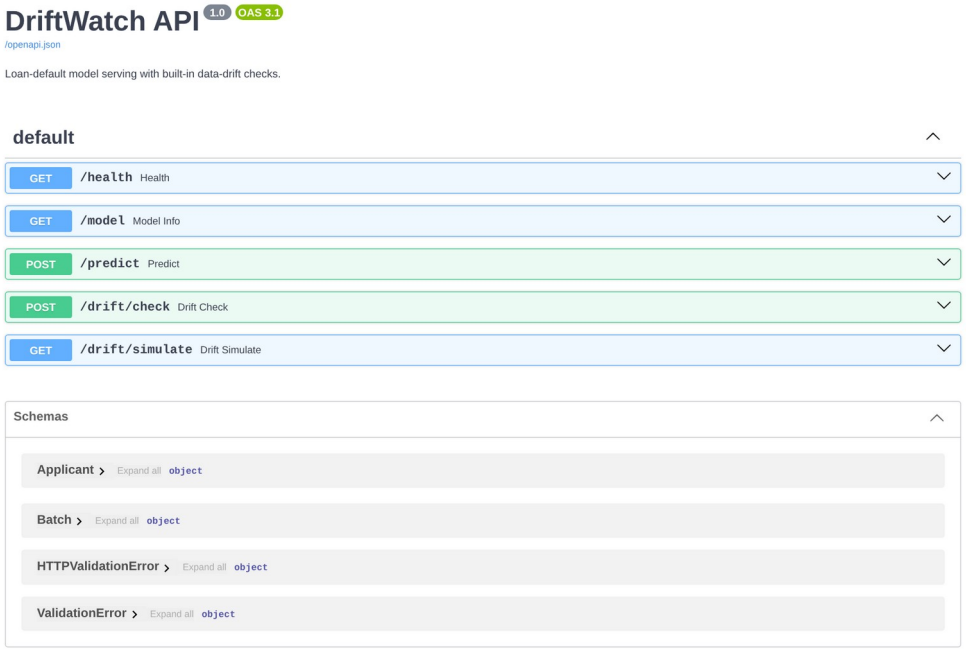


Figure 4.3: FastAPI service documentation (Swagger UI)

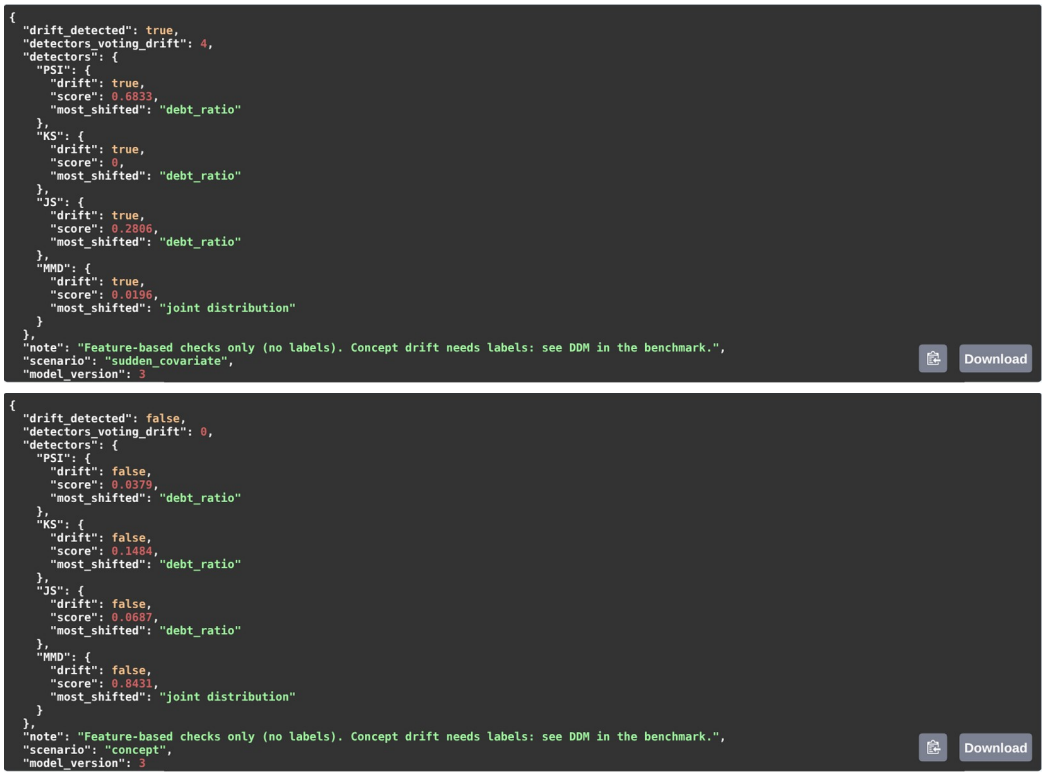


Figure 4.4: /drift/simulate responses: sudden covariate drift is flagged (top), concept drift is not (bottom)

4.1.6 Testing and Continuous Integration

Six automated tests check that the feature detectors flag an obvious shift, that the threshold detectors stay quiet on identical distributions, that DDM reacts to a rise in error, that the drift schedule is correct, and that the full pipeline recovers from concept drift. A GitHub Actions

workflow runs on every push. It installs dependencies, runs the tests, executes a two-seed benchmark, and applies an **accuracy quality gate**: the build fails if DDM-triggered retraining no longer improves concept-drift accuracy by at least ten percentage points over no retraining. The benchmark results are uploaded as a build artifact. A Dockerfile packages the API, with a trained model baked in, into a container image.

4.2 Results and Discussion

All results are means over five random seeds. The full benchmark ran in about three minutes on a two-core machine.

4.2.1 Accuracy over Time

Figure 4.5 shows mean accuracy per batch for four representative strategies. In the no-drift control, all strategies stay near 0.87, confirming that unnecessary retraining neither helps nor harms much. Covariate drift lowers accuracy only modestly, from 0.876 to about 0.845, and all lines remain close together. Concept drift is dramatically different: accuracy without retraining collapses to about 0.63 and never recovers. PSI never fires, so its line coincides with no retraining. DDM recovers within three batches, while periodic retraining recovers only at its next scheduled run, batch 35.



Figure 4.5: Mean model accuracy per batch under the four drift scenarios

4.2.2 Detection Speed and Reliability

Table 4.4 and Figure 4.6 summarise detection performance. All four feature-based detectors caught sudden covariate drift in every run, in the very first drifted batch. For gradual drift, the

sensitive KS test was fastest (1.4 batches), followed by MMD (3.4), JS (4.0) and PSI (8.6), whose fixed 0.2 threshold is reached only once the shift is well advanced.

For concept drift the picture reverses. PSI and JS never detected it, because the input distribution does not change. KS and MMD appear to "detect" it in 40% and 20% of runs, but these alarms occur at the same rate as their false alarms in the no-drift control (6.0 and 3.7 per 100 batches), so they are chance hits rather than genuine detections. Only DDM detected concept drift reliably: in 100% of runs, with a mean delay of 0.2 batches. DDM caught covariate drift less often (60% sudden, 40% gradual), because that drift barely raised the model's error. As the next section shows, this was the correct behaviour.

Table 4.4: Detection performance (detection rate / mean delay in batches) and false alarms

Strategy	Sudden covariate	Gradual covariate	Concept	False alarms /100 batches
Periodic	100% / 5.0	100% / 5.0	100% / 5.0	10.0
PSI	100% / 0.0	100% / 8.6	missed	0.0
KS	100% / 0.0	100% / 1.4	40% / 1.5	6.0
JS	100% / 0.0	100% / 4.0	missed	0.0
MMD	100% / 0.0	100% / 3.4	20% / 8.0	3.7
DDM	60% / 5.0	40% / 9.5	100% / 0.2	0.0

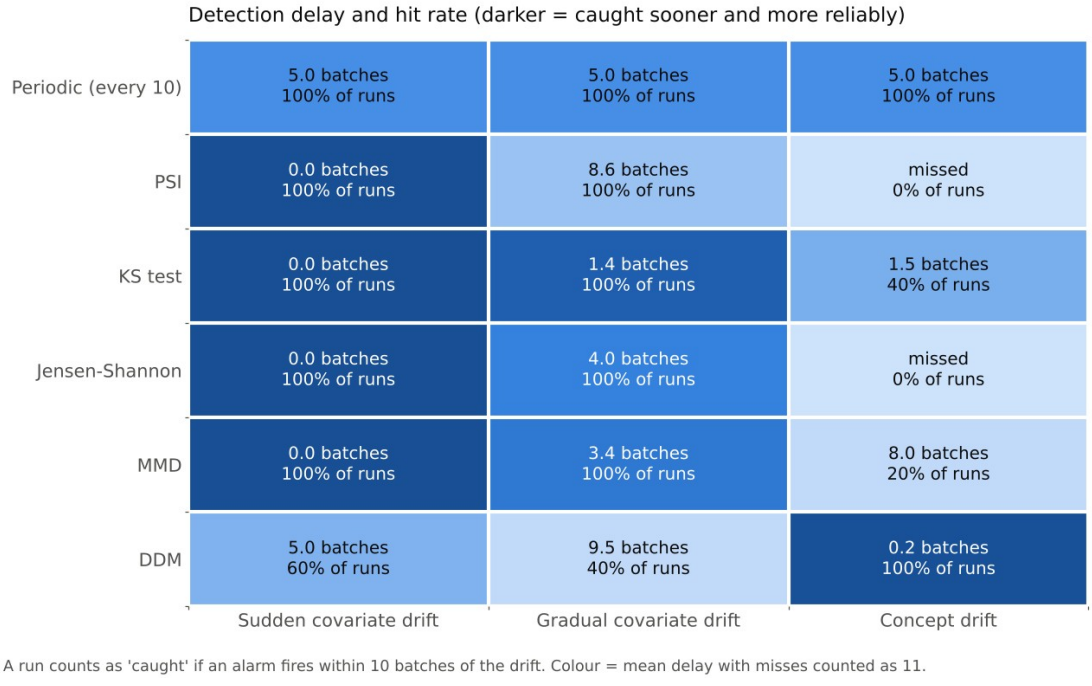


Figure 4.6: Detection delay and hit rate per detector and scenario

4.2.3 Accuracy after Drift and Cost of Retraining

Table 4.5 and Figure 4.7 report post-drift accuracy. Under concept drift, DDM achieved the highest accuracy (0.833) and F1-score (0.745), compared with 0.629 and 0.381 without

retraining. That is an improvement of 20.4 accuracy points, achieved with only two retraining jobs per stream. Periodic retraining reached 0.808, but at the cost of six retrains per stream whether or not drift occurred. Under covariate drift, the best strategy improved accuracy by less than one point over never retraining (0.852 vs 0.845), even though every feature detector raised alarms. This illustrates a key practical point: a statistically significant change in the inputs does not necessarily harm the model, so retraining on every distribution alarm can waste resources.

Figure 4.8 shows the cost side in the no-drift control. PSI, JS and DDM never retrained. KS and MMD triggered 7.2 and 4.4 retraining jobs per stream, including follow-ups, and the periodic baseline triggered 6, all of them unnecessary. In a real system each of these is a training job, a validation run and a potential deployment.

Table 4.5: Mean accuracy after drift per scenario, concept-drift F1-score and wasted retrains

Strategy	No drift	Sudden cov.	Gradual cov.	Concept	F1 (concept)	Retrains (no drift)
No retraining	0.876	0.845	0.849	0.629	0.381	0.0
Periodic	0.873	0.848	0.850	0.808	0.704	6.0
PSI	0.876	0.848	0.851	0.629	0.381	0.0
KS	0.872	0.852	0.851	0.743	0.588	7.2
JS	0.876	0.852	0.852	0.629	0.381	0.0
MMD	0.875	0.851	0.852	0.693	0.491	4.4
DDM	0.876	0.848	0.851	0.833	0.745	0.0

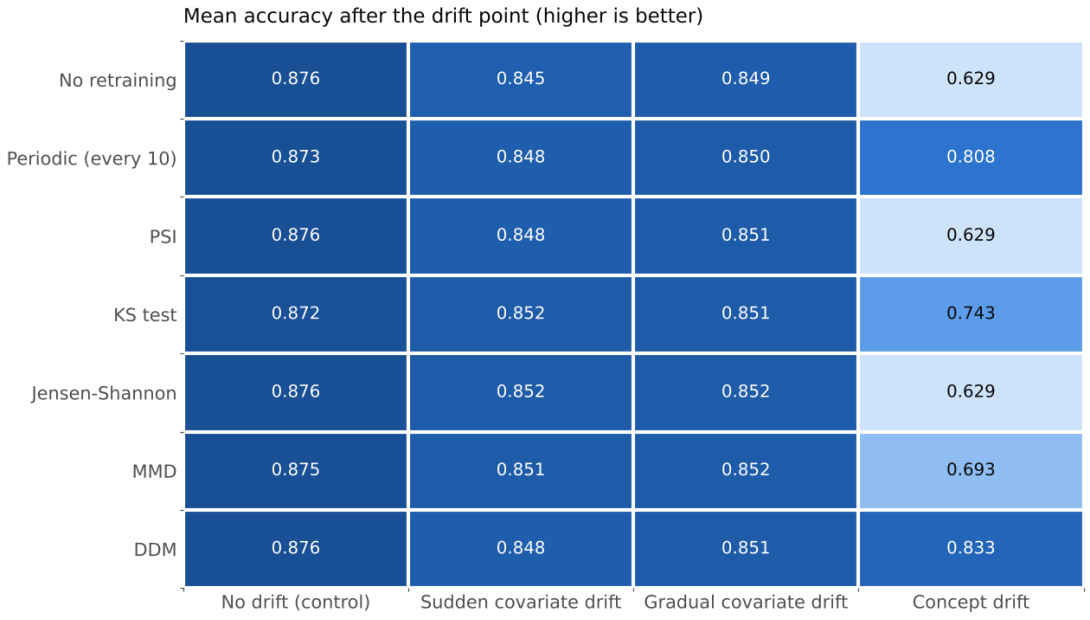


Figure 4.7: Mean accuracy after the drift point for each strategy and scenario

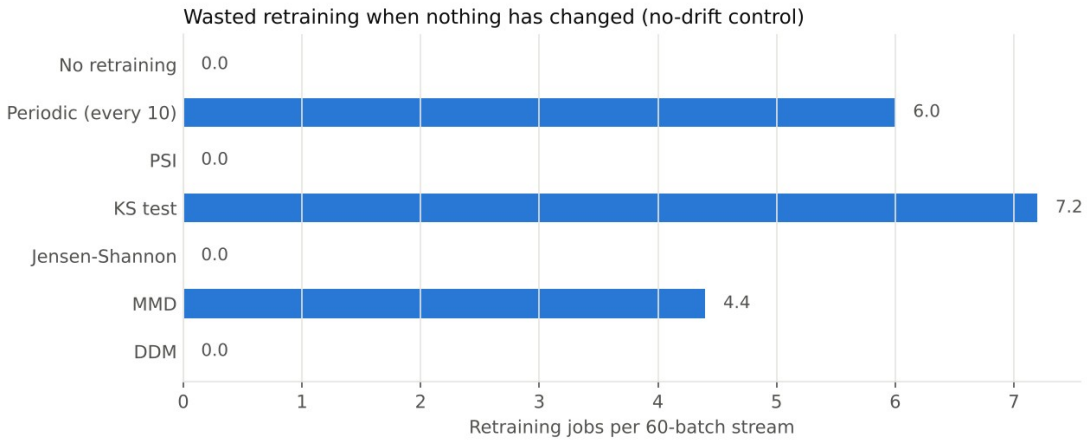


Figure 4.8: Unnecessary retraining jobs per stream in the no-drift control

4.2.4 Discussion

The results confirm that the two detector families answer different questions. Distribution-based tests answer “has the input changed?”. They are cheap, need no labels and give early warning, but they are blind to concept drift and can over-react to harmless covariate shift. Performance-based monitoring answers “is the model getting worse?”, which is the question that justifies retraining, but it depends on labels being available. Among the distribution tests, PSI and JS offered the best balance for alerting, with no false alarms, while KS and MMD were faster on gradual drift at the cost of frequent false alarms. The champion/challenger gate proved its value throughout: across all 140 runs, a challenger was promoted only when it matched or beat the champion on the newest data, so no retraining event could push a worse model live.

These findings suggest a **hybrid monitoring design**: run PSI or JS continuously as a label-free early-warning signal; use DDM or a similar performance monitor, once labels arrive, as the trigger for retraining; and keep a champion/challenger quality gate in front of every promotion.

The study has limitations. The data is synthetic, which gives exact control over drift onset but may not capture every pattern seen in real data. Labels are assumed to arrive immediately after each batch, whereas in practice they may be delayed by weeks. Thresholds were set to standard defaults rather than tuned per detector.

4.3 Individual Contribution

This was an individual project. All work was carried out by the author: the literature review; the design of the drift scenarios and the retraining policy; the implementation of the detectors, pipeline, benchmark, API, tests and CI workflow; running the experiments; and the analysis and writing of this report.

CHAPTER 5
CONCLUSION AND FUTURE WORK

This project built DriftWatch, a complete MLOps pipeline that serves a model, monitors it for drift, retrains automatically and promotes new models only through a champion/challenger quality gate. It is backed by MLflow tracking and model registry, a FastAPI service, automated tests and CI. Using this pipeline, five drift detectors and two baselines were compared over 140 simulated production streams with controlled drift.

The main conclusions are:

1. **Distribution-based detectors cannot see concept drift.** PSI and JS never detected it, leaving accuracy at 62.9%, the same as never retraining.
2. **Performance-based detection is the most reliable retraining trigger.** DDM detected concept drift in every run within 0.2 batches, with zero false alarms, and achieved the best post-drift accuracy (0.833) using two retrains per stream.
3. **Not all drift is harmful.** Covariate drift was flagged instantly by every feature detector, but retraining improved accuracy by less than one point.
4. **Sensitivity has a cost.** KS and MMD caused 7.2 and 4.4 unnecessary retraining jobs per stream with no drift present, and periodic retraining caused 6.
5. **A hybrid design is recommended**: label-free distribution tests for early warning, performance monitoring to trigger retraining, and a champion/challenger gate before every promotion.

Future work will extend the study to real data with naturally occurring drift, delayed labels, additional detectors and the cost of retraining in compute and carbon terms.

5.1 Proposed Work Plan of the Project

Table 5.1 sets out the proposed plan for extending this work.

Table 5.1: Proposed work plan

Phase	Activity	Duration
1	Validate on a real, time-stamped dataset (e.g. Lending Club loans) where drift arises naturally	2 weeks
2	Model delayed labels, so DDM only sees outcomes weeks after prediction, and measure the effect on recovery	1 week
3	Implement and evaluate the hybrid policy (PSI/JS warning + DDM trigger) as an eighth strategy	1 week
4	Add ADWIN and EDDM detectors, and tune each detector’s threshold for a fixed false-alarm budget	1 week
5	Track compute and carbon cost per retraining job (CodeCarbon) and add it to the cost analysis	1 week
6	Deploy the API and a scheduled monitoring job to the cloud, with alerting through a dashboard	2 weeks

REFERENCES

Journal / Conference Papers and Web Resources (IEEE format, numbered in order of first citation)

[1] D. Sculley, G. Holt, D. Golovin, E. Davydov, T. Phillips, D. Ebner, V. Chaudhary, M. Young, J.-F. Crespo, and D. Dennison, "Hidden technical debt in machine learning systems," in *Advances in Neural Information Processing Systems 28 (NeurIPS)*, 2015, pp. 2503–2511.

[2] D. Kreuzberger, N. Kühl, and S. Hirschl, "Machine learning operations (MLOps): Overview, definition, and architecture," *IEEE Access*, vol. 11, pp. 31866–31879, 2023.

[3] A. Paleyes, R.-G. Urma, and N. D. Lawrence, "Challenges in deploying machine learning: A survey of case studies," *ACM Computing Surveys*, vol. 55, no. 6, Art. 114, 2022.

[4] J. Gama, I. Žliobaitė, A. Bifet, M. Pechenizkiy, and A. Bouchachia, "A survey on concept drift adaptation," *ACM Computing Surveys*, vol. 46, no. 4, Art. 44, 2014.

[5] J. Lu, A. Liu, F. Dong, F. Gu, J. Gama, and G. Zhang, "Learning under concept drift: A review," *IEEE Transactions on Knowledge and Data Engineering*, vol. 31, no. 12, pp. 2346–2363, 2019.

[6] J. G. Moreno-Torres, T. Raeder, R. Alaiz-Rodríguez, N. V. Chawla, and F. Herrera, "A unifying view on dataset shift in classification," *Pattern Recognition*, vol. 45, no. 1, pp. 521–530, 2012.

[7] J. Gama, P. Medas, G. Castillo, and P. Rodrigues, "Learning with drift detection," in *Advances in Artificial Intelligence – SBIA 2004*, LNCS, vol. 3171, Springer, 2004, pp. 286–295.

[8] A. Bifet and R. Gavaldà, "Learning from time-changing data with adaptive windowing," in *Proc. SIAM International Conference on Data Mining*, 2007, pp. 443–448.

[9] A. Gretton, K. M. Borgwardt, M. J. Rasch, B. Schölkopf, and A. Smola, "A kernel two-sample test," *Journal of Machine Learning Research*, vol. 13, pp. 723–773, 2012.

[10] S. Rabanser, S. Günnemann, and Z. C. Lipton, "Failing loudly: An empirical study of methods for detecting dataset shift," in *Advances in Neural Information Processing Systems 32 (NeurIPS)*, 2019.

[11] B. Yurdakul, "Statistical properties of population stability index," Ph.D. dissertation, Western Michigan University, Kalamazoo, MI, USA, 2018.

[12] F. J. Massey Jr., "The Kolmogorov-Smirnov test for goodness of fit," *Journal of the American Statistical Association*, vol. 46, no. 253, pp. 68–78, 1951.

[13] J. Lin, "Divergence measures based on the Shannon entropy," *IEEE Transactions on Information Theory*, vol. 37, no. 1, pp. 145–151, 1991.

[14] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[15] M. Zaharia, A. Chen, A. Davidson, A. Ghodsi, S. A. Hong, A. Konwinski, et al., "Accelerating the machine learning lifecycle with MLflow," *IEEE Data Engineering Bulletin*, vol. 41, no. 4, pp. 39–45, 2018.

[16] G. I. Webb, R. Hyde, H. Cao, H. L. Nguyen, and F. Petitjean, "Characterizing concept drift," *Data Mining and Knowledge Discovery*, vol. 30, no. 4, pp. 964–994, 2016.

[17] E. Breck, S. Cai, E. Nielsen, M. Salib, and D. Sculley, "The ML test score: A rubric for ML production readiness and technical debt reduction," in *Proc. IEEE International Conference on Big Data*, 2017, pp. 1123–1132.

[18] Evidently AI, "Data drift: Evidently documentation," docs.evidentlyai.com. Last accessed: Oct. 25, 2026.

[19] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[20] M. Baena-García, J. del Campo-Ávila, R. Fidalgo, A. Bifet, R. Gavaldà, and R. Morales-Bueno, “Early drift detection method,” in *Proc. 4th International Workshop on Knowledge Discovery from Data Streams*, 2006.

[21] Google Cloud, “MLOps: Continuous delivery and automation pipelines in machine learning,” cloud.google.com. Last accessed: Oct. 25, 2026.

ANNEXURES

Annexure A: Source Code, Screenshots and Run Instructions

A.1 Source code repository: https://github.com/anu091104/driftwatch. The repository contains the full source code, tests, CI workflow, Dockerfile, benchmark results (CSV files, charts and an HTML report) and a README with run instructions.

A.2 How to run (Windows, VS Code terminal):

```
python -m venv .venv
.venv\Scripts\Activate.ps1
pip install -r requirements.txt
python -m src.demo                    # live demo
python -m src.experiment              # full 140-run benchmark + report
uvicorn api.main:app --reload         # API at http://127.0.0.1:8000/docs
mlflow ui --backend-store-uri sqlite:///mlflow.db   # http://127.0.0.1:5000
pytest -q                             # tests
```

A.3 Additional screenshots:

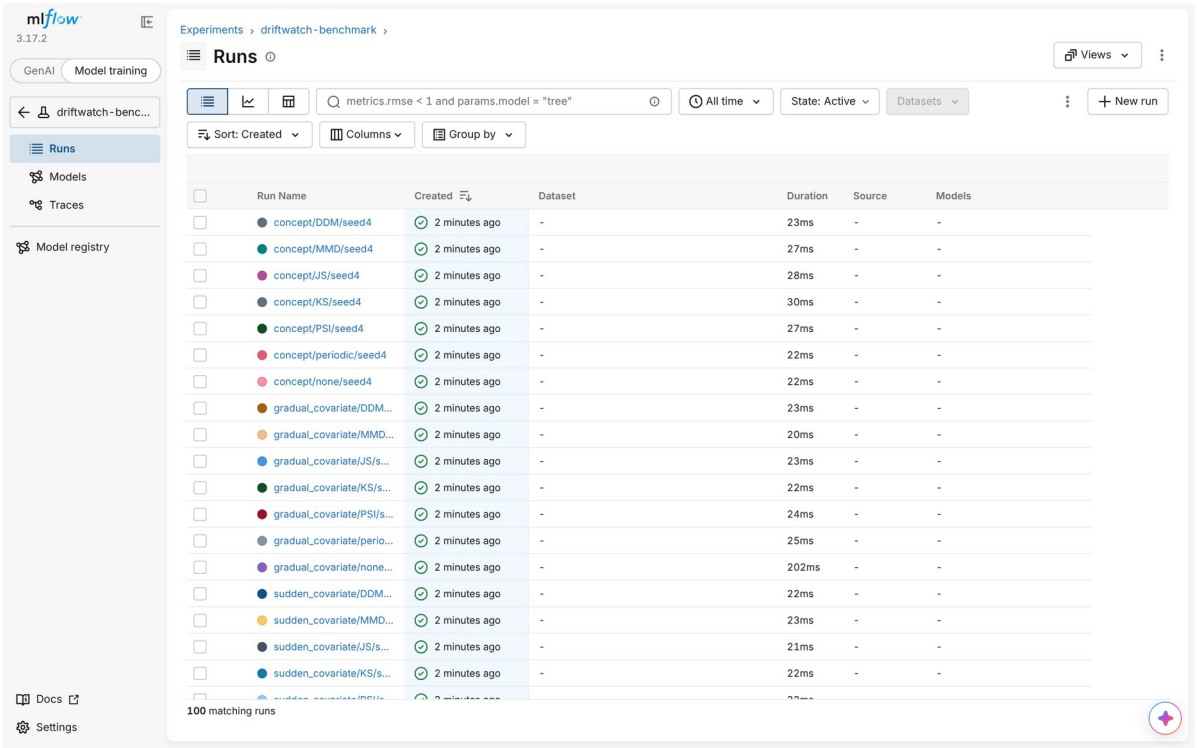


Figure A.1: MLflow runs table for the driftwatch-benchmark experiment

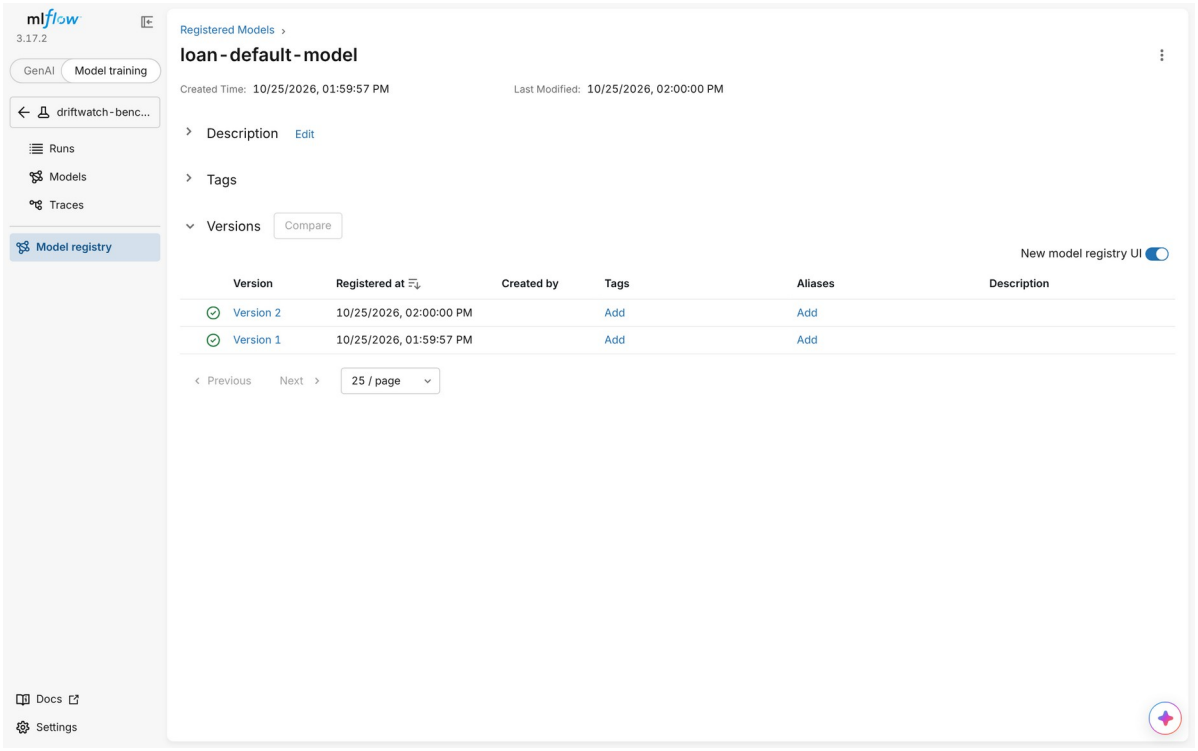


Figure A.2: MLflow Model Registry: promoted models registered as versions of loan-default-model